

Equal-tailed item-restscore p-values under the null

Null check of `RMitemRestscore()` after the switch from `|t|` to equal tails

Magnus Johansson

2026-10-02

Table of contents

Purpose	1
Design	2
Cells and data generation	2
One dataset	3
Run	4
Reproduction of the validation study	5
Results	6
Family-wise rate	6
Per-item rejection by direction	7
Marginal p-values	7
Synthetic nulls	8
Interpretation	11
Follow-up: two one-sided step-downs	11
Checks	14
Family-wise rate and tail balance	14
Balance across items in the mixed arm	16
Pooled moments and a second seed block	17
Real restscore null for the split candidates	23
Checks	26
Family-wise rate and tail balance	27
Paired comparison	28
Balance across items	29

Purpose

`restscore_cutoff_validation.qmd` validated the bootstrap item-restscore test with p-values from `|t|`, the difference studentised by the bootstrap mean and SD and folded at the mean. The marginal level was right, but the null of the difference is left-skewed, so the underfit tail took 2.6 to 3.4 percent per item and the overfit tail 1.8 to 2.7. `RMitemRestscore()` now uses equal-tailed p-values instead:

- **Marginal:** twice the smaller one-sided Monte Carlo p-value, so the smallest attainable value is $2/(B + 1)$.
- **Westfall-Young:** each item's difference is mapped through its own simulated distribution to a normal score, and the max step-down runs on the absolute normal scores. This is a rank-based (min-p) form of the step-down, so it should hold the family-wise rate under the same exchangeability that the `|t|` version relies on.

This study checks that claim by simulation, and that the two directions now get $\alpha/2$ each.

Design

The four cells and 500 datasets per cell of the validation study, with the conditional DGP (now the default) and $B = 400$. The seeds are the same, so each dataset and each bootstrap simulation is identical to the conditional arm of the validation study. Both p-values are computed from the same simulated values:

- `eq`: equal-tailed, as `RMitemRestscore()` now computes it.
- `abs`: $|t|$, as before the change. It should reproduce the validation study's cached results exactly, which checks that nothing else changed.

Outcomes. Family-wise rate (WY and BH) for both, nominal 5 percent with a Monte Carlo SE of 0.97 points at 500 datasets. Per-item marginal rejection by direction, nominal 2.5 percent each. The distribution of marginal p-values.

Cost. The conditional arm of the validation study took 132 minutes on 10 cores over the four cells. Results are saved per cell, so an interrupted render resumes. Set `R_DATASETS` lower for a quick run.

```
suppressMessages({
  devtools::load_all("../", quiet = TRUE)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
options(rgl.useNULL = TRUE)

# The run needed tail = "equal" in .bootstrap_pvalues(), which has since
# been withdrawn. The cached results below come from that version; rendering
# without the cache would fail in one_dataset().

R_DATASETS <- 500
B <- 400
THETA_SD <- 1.5
NCORES <- 10
RES_DIR <- "restscore_eti_null"
VAL_DIR <- "restscore_cutoff_validation"
dir.create(RES_DIR, showWarnings = FALSE)
```

Cells and data generation

Copied from the validation study. The seed of a dataset is $1e6 * (\text{row of its cell}) + \text{rep}$, so the cell order must not change.

```
item_sets <- list(
  dich = as.list(seq(-2, 2, length.out = 20)),
  poly = lapply(seq(-1.5, 1.5, length.out = 9), function(l) l + c(-1.2, 0, 1.2))
)

cells <- data.frame(
  cell = c("dich_mis_150", "dich_tgt_300", "poly_tgt_150", "poly_tgt_1000"),
  set = c("dich", "dich", "poly", "poly"),
  mu = c(1.5, 0, 0, 0),
  n = c(150, 300, 150, 1000),
  stringsAsFactors = FALSE
)
cells
```

cell	set	mu	n
dich_mis_150	dich	1.5	150
dich_tgt_300	dich	0.0	300
poly_tgt_150	poly	0.0	150
poly_tgt_1000	poly	0.0	1000

```
sim_pcm <- function(theta, thr) {
  n <- length(theta)
  sapply(thr, function(d) {
    m <- length(d)
    eta <- outer(theta, 0:m) -
      matrix(c(0, cumsum(d)), n, m + 1, byrow = TRUE)
    p <- exp(eta - apply(eta, 1, max))
    p <- p / rowSums(p)
    cp <- p %*% upper.tri(diag(m + 1), diag = TRUE)
    rowSums(runif(n) > cp)
  })
}
```

One dataset

```
one_dataset <- function(rep, cell_row) {
  thr <- item_sets[[cell_row$set]]
  seed <- 1e6 * match(cell_row$cell, cells$cell) + rep
  set.seed(seed)
  repeat {
    d <- as.data.frame(sim_pcm(rnorm(cell_row$n, cell_row$mu, THETA_SD), thr))
    names(d) <- paste0("I", seq_along(thr))
    full <- all(mapply(function(x, m) all(tabulate(x + 1, m + 1) > 0),
                      d, lengths(thr)))
    if (full) break
  }

  tryCatch({
    co <- suppressMessages(RMitemRestscoreCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile", dgp = "conditional"
    ))
    # The package path, equal-tailed.
    run <- function(correction) {
      suppressMessages(RMitemRestscore(
        d, cutoff = co, correction = correction, output = "dataframe"
      ))
    }
    eq_wy <- run("fwer")
    eq_bh <- run("fdr_bh")
    eq_none <- run("none")

    # |t| from the same simulated values, as RMitemRestscore() computed it
    # before the change.
    sim_mat <- tapply(co$results$Difference,
                      list(co$results$iteration, co$results$Item),
```

```

        function(x) x[1L])
obs <- stats::setNames(eq_wy$Difference, eq_wy$Item)
abs_p <- function(correction) {
  pv <- .bootstrap_pvalues(obs, sim_mat, correction = correction,
                           tail = "two.sided")
  pv[match(names(obs), pv$name), ]
}
a_wy <- abs_p("fwer")
a_bh <- abs_p("fdr_bh")
null_mean <- colMeans(sim_mat, na.rm = TRUE)[names(obs)]

data.frame(
  rep = rep,
  item = eq_wy$Item,
  difference = eq_wy$Difference,
  p_eq = eq_wy$p_restscore,
  padj_eq_wy = eq_wy$padj_restscore,
  padj_eq_bh = eq_bh$padj_restscore,
  flag_eq_marginal = eq_none$Flagged,
  p_abs = a_wy$p,
  padj_abs_wy = a_wy$padj,
  padj_abs_bh = a_bh$padj,
  flag_abs_marginal = ifelse(
    a_wy$p < .05, ifelse(obs > null_mean, "overfit", "underfit"), ""
  ),
  actual_iterations = co$actual_iterations,
  stringsAsFactors = FALSE
)
}, error = function(e) {
  data.frame(rep = rep, error = conditionMessage(e))
})
}

```

Errors are kept as rows, so a failure shows in the results instead of looking like a missing dataset.

Run

```

for (i in seq_len(nrow(cells))) {
  f <- file.path(RES_DIR, paste0(cells$cell[i], ".rds"))
  if (file.exists(f)) next
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_DATASETS),
                 function(r) one_dataset(r, cells[i, ]),
                 mc.cores = NCORES, mc.preschedule = FALSE)
  saveRDS(list(res = res,
              elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
          f)
}

raw <- lapply(cells$cell, function(cl) {
  x <- readRDS(file.path(RES_DIR, paste0(cl, ".rds")))
  is_ok <- vapply(x$res, function(r) "item" %in% names(r), logical(1))
  list(ok = cbind(cell = cl, do.call(rbind, x$res[is_ok])),
       err = if (any(!is_ok)) cbind(cell = cl, do.call(rbind, x$res[!is_ok])),

```

```

      minutes = round(x$elapsed_min, 1))
    })
    long <- do.call(rbind, lapply(raw, `[`, "ok"))
    long$cell <- factor(long$cell, levels = cells$cell)
    errors <- do.call(rbind, lapply(raw, `[`, "err"))

    data.frame(
      cell = cells$cell,
      datasets = vapply(raw, function(x) length(unique(x$ok$rep)), integer(1)),
      errors = vapply(raw, function(x) if (is.null(x$err)) 0L else nrow(x$err),
                      integer(1)),
      minutes = vapply(raw, `[`, numeric(1), "minutes")
    )

```

cell	datasets	errors	minutes
dich_mis_150	500	0	25.8
dich_tgt_300	500	0	32.8
poly_tgt_150	500	0	26.2
poly_tgt_1000	500	0	47.8

```

if (!is.null(errors)) count(errors, cell, error)

```

Reproduction of the validation study

The t p-values should equal the cached conditional arm of the validation study for every dataset both have.

```

val <- do.call(rbind, lapply(cells$cell, function(cl) {
  x <- readRDS(file.path(VAL_DIR, paste0(cl, "_conditional.rds")))
  cbind(cell = cl, do.call(rbind, Filter(Negate(is.null), x$res)))
}))
chk <- inner_join(
  long |> mutate(cell = as.character(cell)) |>
    select(cell, rep, item, difference, p_abs, padj_abs_wy, padj_abs_bh),
  val |> select(cell, rep, item, difference_val = difference, p_val = p,
               padj_fwer_val = padj_fwer, padj_bh_val = padj_bh),
  by = c("cell", "rep", "item")
)
chk |>
  group_by(cell) |>
  summarise(items = n(),
            max_diff_difference = max(abs(difference - difference_val)),
            max_diff_p = max(abs(p_abs - p_val)),
            max_diff_padj_wy = max(abs(padj_abs_wy - padj_fwer_val)),
            max_diff_padj_bh = max(abs(padj_abs_bh - padj_bh_val)),
            .groups = "drop")

```

cell	items	max_diff_difference	max_diff_p	max_diff_padj_wy	max_diff_padj_bh
dich_mis_150	10000	0	0	0	0
dich_tgt_300	10000	0	0	0	0

cell	items	max_diff_difference	max_diff_p	max_diff_padj_wy	max_diff_padj_bh
poly_tgt_1000	4500	0	0	0	0
poly_tgt_150	4500	0	0	0	0

Results

Family-wise rate

Datasets with at least one item flagged at $\alpha = .05$.

```
fw <- long |>
  group_by(cell, rep) |>
  summarise(eq_wy = any(padj_eq_wy < .05, na.rm = TRUE),
            abs_wy = any(padj_abs_wy < .05, na.rm = TRUE),
            eq_bh = any(padj_eq_bh < .05, na.rm = TRUE),
            abs_bh = any(padj_abs_bh < .05, na.rm = TRUE),
            .groups = "drop")
fw |>
  group_by(cell) |>
  summarise(datasets = n(),
            across(c(eq_wy, abs_wy, eq_bh, abs_bh), \(x) round(100 * mean(x), 1)),
            .groups = "drop")
```

cell	datasets	eq_wy	abs_wy	eq_bh	abs_bh
dich_mis_150	500	8.8	4.2	0.4	2.8
dich_tgt_300	500	11.0	6.8	1.0	6.2
poly_tgt_150	500	6.2	3.4	2.0	2.8
poly_tgt_1000	500	9.4	5.6	4.8	5.0

```
fw |>
  summarise(datasets = n(),
            across(c(eq_wy, abs_wy, eq_bh, abs_bh), \(x) round(100 * mean(x), 2)))
```

datasets	eq_wy	abs_wy	eq_bh	abs_bh
2000	8.85	5	2.05	4.2

Paired: datasets flagged by WY under one p-value and not the other.

```
fw |>
  group_by(cell) |>
  summarise(eq_only = sum(eq_wy & !abs_wy), abs_only = sum(!eq_wy & abs_wy),
            both = sum(eq_wy & abs_wy), .groups = "drop")
```

cell	eq_only	abs_only	both
dich_mis_150	33	10	11
dich_tgt_300	25	4	30
poly_tgt_150	15	1	16
poly_tgt_1000	22	3	25

Per-item rejection by direction

Marginal $p < .05$, split by direction, nominal 2.5 percent each. `se` is the Monte Carlo SE with datasets as clusters.

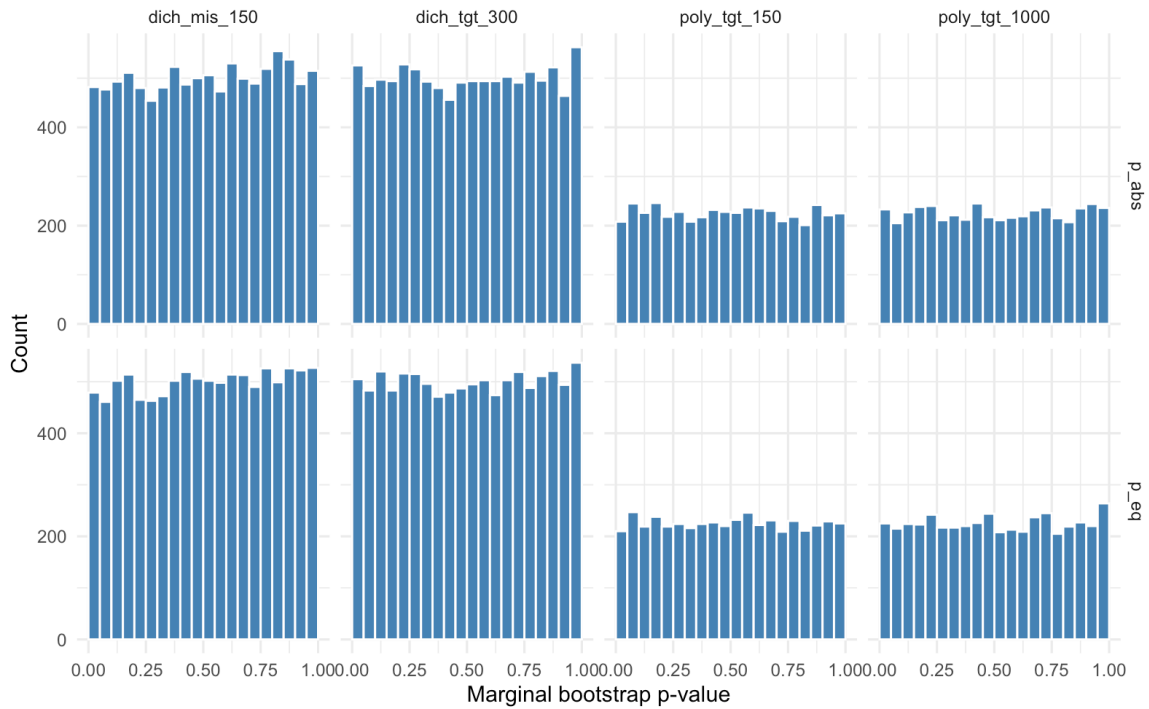
```
long |>
  group_by(cell, rep) |>
  summarise(eq_over = mean(flag_eq_marginal == "overfit"),
            eq_under = mean(flag_eq_marginal == "underfit"),
            abs_over = mean(flag_abs_marginal == "overfit"),
            abs_under = mean(flag_abs_marginal == "underfit"),
            .groups = "drop") |>
  group_by(cell) |>
  summarise(across(c(eq_over, eq_under, abs_over, abs_under),
                    list(pct = \(\x) round(100 * mean(x), 2),
                         se = \(\x) round(100 * sd(x) / sqrt(n()), 2))),
            .groups = "drop")
```

cell	eq_over_pct	eq_over_se	eq_under_pct	eq_under_se	abs_over_pct	abs_over_se	abs_under_pct	abs_under_se
dich_mis_150	2.57	0.15	2.21	0.14	1.77	0.13	3.04	0.16
dich_tgt_300	2.52	0.15	2.53	0.14	2.21	0.14	3.05	0.16
poly_tgt_150	2.29	0.22	2.33	0.21	1.98	0.21	2.64	0.23
poly_tgt_1000	2.42	0.21	2.58	0.21	2.40	0.21	2.78	0.22

Marginal p-values

The equal-tailed p-value takes the values $2k/(B + 1)$, so its histogram is coarser than the $|t|$ one. Both should be flat.

```
long |>
  select(cell, p_eq, p_abs) |>
  pivot_longer(c(p_eq, p_abs), names_to = "pvalue", values_to = "p") |>
  ggplot(aes(p)) +
  geom_histogram(breaks = seq(0, 1, 0.05), fill = "steelblue",
                 colour = "white") +
  facet_grid(pvalue ~ cell, scales = "free_y") +
  labs(x = "Marginal bootstrap p-value", y = "Count") +
  theme_minimal()
```



Synthetic nulls

The family-wise excess could come from the restscore bootstrap or from the step-down itself. To separate the two, the variants below are run on nulls where the observed vector is exchangeable with the simulated ones by construction: $B + 1$ independent rows, the first taken as observed. Each column is a left-skewed variable, $-\exp(s * Z)$, with Z standard normal and correlated ρ across columns. 4000 replications per condition (Monte Carlo SE about 0.35 points at 5 percent), $B = 400$.

The equal-tailed code has been removed from the package, so the variants are defined here.

- **first**: the version that ran above. Simulated values are scored by their mid-rank among the simulated values, the observed value by its mid-rank against them.
- **pooled**: observed and simulated values ranked together, $B + 1$ per item, $qnorm((rank - 0.5) / (B + 1))$.
- **pooled_sym**: as **pooled**, but $|z|$ computed from the rank counted from the nearer end, so the two tails give identical magnitudes.
- **two_piece**: continuous. The difference from the simulated median, scaled by (median - 2.5th percentile) below it and (97.5th percentile - median) above it.
- **abs_t**: the package's two-sided $|t|$.

```
SYN_FILE <- file.path(RES_DIR, "synthetic.rds")
if (!file.exists(SYN_FILE)) {
  z_first <- function(o, sim) {
    sc <- function(x, ref) {
      ref <- sort(ref)
      b <- findInterval(x, ref, left.open = TRUE)
      a <- findInterval(x, ref)
      stats::qnorm((0.5 * (b + a) + 0.5) / (length(ref) + 1))
    }
    list(o = sc(o, sim), s = sc(sim, sim))
  }
  z_pooled <- function(o, sim, sym) {
    r <- rank(c(o, sim))
    n <- length(r)
```

```

z <- if (sym) {
  sign(r - (n + 1) / 2) * -stats::qnorm((pmin(r, n + 1 - r) - 0.5) / n)
} else {
  stats::qnorm((r - 0.5) / n)
}
list(o = z[1], s = z[-1])
}
z_two_piece <- function(o, sim) {
  q <- stats::quantile(sim, c(.025, .5, .975), names = FALSE)
  f <- function(v) ifelse(v < q[2], (v - q[2]) / (q[2] - q[1]),
    (v - q[2]) / (q[3] - q[2]))
  list(o = f(o), s = f(sim))
}
z_abs_t <- function(o, sim) {
  m <- mean(sim); sdv <- stats::sd(sim)
  list(o = (o - m) / sdv, s = (sim - m) / sdv)
}
variants <- list(
  first = z_first,
  pooled = function(o, sim) z_pooled(o, sim, FALSE),
  pooled_sym = function(o, sim) z_pooled(o, sim, TRUE),
  two_piece = z_two_piece,
  abs_t = z_abs_t
)
one_syn <- function(r, k, rho, skew, B = 400) {
  set.seed(r)
  common <- rnorm(B + 1)
  x <- sapply(seq_len(k), function(j) {
    -exp(skew * (sqrt(rho) * common + sqrt(1 - rho) * rnorm(B + 1)))
  })
  x <- matrix(x, B + 1, k)
  unlist(lapply(variants, function(v) {
    zz <- lapply(seq_len(k), function(j) v(x[1, j], x[-1, j]))
    zo <- vapply(zz, `[`, numeric(1), "o")
    zs <- sapply(zz, `[`, "s")
    zs <- matrix(zs, B, k)
    padj <- .wy_stepdown(abs(zo), abs(zs), B)
    c(any = any(padj < .05), low = sum(padj < .05 & zo < 0),
      high = sum(padj < .05 & zo > 0))
  })))
}
grid <- expand.grid(k = c(1, 9, 20), rho = c(0, 0.5), skew = c(0.5, 0.15))
syn <- do.call(rbind, lapply(seq_len(nrow(grid)), function(g) {
  res <- do.call(rbind, mclapply(seq_len(4000), function(r) {
    one_syn(r, grid$k[g], grid$rho[g], grid$skew[g])
  }, mc.cores = NCORES))
  cbind(grid[g, ], as.data.frame(t(colMeans(res[, grepl("any", colnames(res))])
* 100)),
    as.data.frame(t(colSums(res[, !grepl("any", colnames(res))])))
})))
saveRDS(syn, SYN_FILE)
}
syn <- readRDS(SYN_FILE)
names(syn) <- sub("\\.", "_", names(syn))

```

```
syn |>
  select(skew, k, rho, ends_with("_any")) |>
  mutate(across(ends_with("_any"), \(x) round(x, 2)))
```

skew	k	rho	first_any	pooled_any	pooled_sym_any	two_piece_any	abs_t_any
0.50	1	0.0	5.15	5.15	5.15	5.25	4.78
0.50	9	0.0	8.65	4.28	4.28	5.38	5.25
0.50	20	0.0	9.22	4.65	0.00	5.55	5.25
0.50	1	0.5	5.27	5.27	5.27	5.42	4.85
0.50	9	0.5	6.75	4.00	3.65	5.05	4.62
0.50	20	0.5	6.88	3.50	0.12	4.80	5.15
0.15	1	0.0	5.15	5.15	5.15	5.27	4.88
0.15	9	0.0	8.65	4.28	4.28	5.70	5.10
0.15	20	0.0	9.22	4.65	0.00	5.88	5.15
0.15	1	0.5	5.27	5.27	5.27	5.42	5.27
0.15	9	0.5	6.75	4.00	3.65	5.25	4.58
0.15	20	0.5	6.88	3.50	0.12	5.78	5.18

```
syn |>
  transmute(skew, k, rho,
    first = paste0(first_low, "/", first_high),
    pooled = paste0(pooled_low, "/", pooled_high),
    pooled_sym = paste0(pooled_sym_low, "/", pooled_sym_high),
    two_piece = paste0(two_piece_low, "/", two_piece_high),
    abs_t = paste0(abs_t_low, "/", abs_t_high))
```

skew	k	rho	first	pooled	pooled_sym	two_piece	abs_t
0.50	1	0.0	89/117	89/117	89/117	92/118	191/0
0.50	9	0.0	174/182	95/78	95/78	219/1	215/0
0.50	20	0.0	195/193	0/193	0/0	230/0	219/0
0.50	1	0.5	100/111	100/111	100/111	105/112	194/0
0.50	9	0.5	174/213	134/110	101/109	317/4	305/0
0.50	20	0.5	208/236	1/228	1/25	369/0	371/0
0.15	1	0.0	89/117	89/117	89/117	92/119	135/60
0.15	9	0.0	174/182	95/78	95/78	178/54	206/2
0.15	20	0.0	195/193	0/193	0/0	202/40	214/2
0.15	1	0.5	100/111	100/111	100/111	104/113	141/70
0.15	9	0.5	174/213	134/110	101/109	213/116	287/15
0.15	20	0.5	208/236	1/228	1/25	267/110	364/6

The first table is the family-wise rate. The second counts WY flags in the lower and upper tail, summed over replications.

Interpretation

1. The marginal equal-tailed test works. Per item, the equal-tailed p-value rejected 2.2 to 2.6 percent in each tail in all four cells, against 1.8 to 3.0 for $|t|$. Splitting α evenly between the tails is achieved at the item level.

2. Its Westfall-Young step-down, as first built, was liberal. At least one item was flagged in 8.85 percent of null datasets (8.8 to 11.0 by cell), against 5.0 percent for $|t|$ on the same simulated values, and the equal-tailed test flagged 95 datasets that $|t|$ did not, against 18 the other way. The $|t|$ results reproduced the validation study exactly, so the difference lies in the p-value alone.

3. The cause is in the step-down, not in the restscore bootstrap. On the synthetic nulls the first version is nominal with one item and liberal with several. Simulated values were scored among themselves, so they can never fall outside the simulated range, while the observed value can. An observed value beyond every simulated value in its own column then beats every simulated maximum, which happens with probability about $2/(B + 1)$ per item. Ranking observed and simulated values together (`pooled`) restores exchangeability.

4. Ranks cannot give a usable step-down at $B = 400$. Once the two tails get identical magnitudes (`pooled_sym`), the test with 20 items almost never rejects. With pooled ranks, each item has one value at each end of its $B + 1$ values, so roughly 2k simulated rows share the largest possible $|z|$, and the smallest attainable adjusted p-value is about $2k/(B + 1)$, near .10 for 20 items. A rank-based step-down needs B well above $2k/\alpha$, 800 iterations for 20 items. The `pooled` version appeared to work only because `qnorm()` gives the two ends magnitudes that differ in the last bits, which broke those ties in favour of the upper tail.

5. A continuous statistic holds the level but not the balance. The two-piece studentisation is continuous, so ties do not arise, and it held the family-wise rate (4.8 to 5.9 percent). But its flags are still lopsided toward the long tail, although less than with $|t|$ at the milder skew (267 against 110 lower and upper flags with 20 correlated items, against 364 and 6 for $|t|$), which makes it the most promising lead for later work. Matching the tails at the 2.5th and 97.5th percentiles does not match them at the far more extreme quantiles where the step-down decides, and estimating those needs many more iterations.

Decision. The equal-tailed p-value was removed from the package on 2026-10-02, and `RMitemRestscore()` keeps the two-sided $|t|$ test validated in `restscore_cutoff_validation.qmd`, with the tail imbalance stated in its help page. An equal-tailed family-wise test is left as an open problem.

Follow-up: two one-sided step-downs

Added 2026-10-02, after the withdrawal, as the first candidate in `HANDOFF-restscore-equal-tails.md`. WY runs twice, once on t (large values in the overfit direction) and once on $-t$, each at $\alpha/2$, and an item is flagged if either run rejects it. The statistic is continuous, so ties do not arise, and observed and simulated values get the same transform, as in $|t|$. Each direction is calibrated against its own one-sided maximum, so its family-wise rate should be $\alpha/2$ whatever the skew, and the total at most α . The critical value is about the 10th largest of the 400 simulated maxima, so $B = 400$ is enough.

Variants, all on the same replications:

- `abs_t`: the shipped two-sided $|t|$ at α . Same seeds and code as the `synthetic` chunk, so it must reproduce the cached results.
- `split_t`: t in the upper run, $-t$ in the lower run, each at $\alpha/2$.
- `split_two_piece`: as `split_t`, but each run scales by its own tail, the upper run by (97.5th percentile – median) and the lower run by (median – 2.5th percentile), both from the simulated values. This is candidate 2 of the handoff. Each run applies one affine transform per item, so only the comparison across items changes.
- `marginal_eq`: the equal-tailed marginal test, for the $k = 1$ check.

In the `synthetic` grid all columns have the same distribution, so scaling per item changes almost nothing and the two split variants should agree. They can differ only when items differ in skew. The

`mixed` arm adds that: the skew parameter rises from 0.02 to 0.3 across items (absolute skewness about 0.06 to 0.95, which brackets the mean of 0.28 in the most skewed restscore cell). Its flags are split by thirds of the items, from least to most skewed.

What to check.

1. `abs_t` reproduces the cached synthetic results.
2. With `k = 1`, `split_t`, `split_two_piece` and `marginal_eq` give the same flag counts.
3. The family-wise rate per direction is 2.5 percent, ± 0.5 (two Monte Carlo SEs at 4000 replications), and the total at or a little below 5.
4. In the `mixed` arm, flags in each direction spread evenly over the thirds. `split_t` should favour the most skewed items in the lower run and the least skewed in the upper run, and `split_two_piece` should reduce that.

About 10 to 15 minutes on 10 cores. Results are cached in `restscore_eti_null/synthetic_split.rds`, so delete it to rerun.

```
SPLIT_FILE <- file.path(RES_DIR, "synthetic_split.rds")
REPS <- 4000
if (!file.exists(SPLIT_FILE)) {
  ALPHA <- 0.05
  # Per item, a statistic for each direction, oriented so that large values
  # are extreme in that direction.
  split_variants <- list(
    split_t = function(o, sim) {
      m <- mean(sim); sdv <- stats::sd(sim)
      to <- (o - m) / sdv; ts <- (sim - m) / sdv
      list(o_hi = to, s_hi = ts, o_lo = -to, s_lo = -ts)
    },
    split_two_piece = function(o, sim) {
      q <- stats::quantile(sim, c(.025, .5, .975), names = FALSE)
      up <- q[3] - q[2]; dn <- q[2] - q[1]
      list(o_hi = (o - q[2]) / up, s_hi = (sim - q[2]) / up,
           o_lo = (q[2] - o) / dn, s_lo = (q[2] - sim) / dn)
    }
  )
  wy_split <- function(v, x, B) {
    k <- ncol(x)
    zz <- lapply(seq_len(k), function(j) v(x[1, j], x[-1, j]))
    obs <- function(nm) vapply(zz, `[`, numeric(1), nm)
    sims <- function(nm) matrix(sapply(zz, `[`, nm), B, k)
    list(low = .wy_stepdown(obs("o_lo"), sims("s_lo"), B) < ALPHA / 2,
         high = .wy_stepdown(obs("o_hi"), sims("s_hi"), B) < ALPHA / 2)
  }
  # The shipped |t|, computed as in the `synthetic` chunk.
  wy_abs_t <- function(x, B) {
    k <- ncol(x)
    zz <- lapply(seq_len(k), function(j) {
      m <- mean(x[-1, j]); sdv <- stats::sd(x[-1, j])
      list(o = (x[1, j] - m) / sdv, s = (x[-1, j] - m) / sdv)
    })
    zo <- vapply(zz, `[`, numeric(1), "o")
    zs <- matrix(sapply(zz, `[`, "s"), B, k)
    padj <- .wy_stepdown(abs(zo), abs(zs), B)
    list(low = padj < ALPHA & zo < 0, high = padj < ALPHA & zo > 0)
  }
}
```

```

# Equal-tailed marginal test, twice the smaller one-sided Monte Carlo p.
marg_eq <- function(x, B) {
  o <- x[1, ]; s <- x[-1, , drop = FALSE]
  p_hi <- (1 + colSums(sweep(s, 2, o, ">="))) / (B + 1)
  p_lo <- (1 + colSums(sweep(s, 2, o, "<="))) / (B + 1)
  list(low = p_lo < ALPHA / 2, high = p_hi < ALPHA / 2)
}

# Same generator and seeds as `synthetic`, with skew allowed to vary by item.
one_rep <- function(r, k, rho, skew, B = 400) {
  set.seed(r)
  common <- rnorm(B + 1)
  s <- rep_len(skew, k)
  x <- sapply(seq_len(k), function(j) {
    -exp(s[j] * (sqrt(rho) * common + sqrt(1 - rho) * rnorm(B + 1)))
  })
  x <- matrix(x, B + 1, k)
  fl <- c(list(abs_t = wy_abs_t(x, B), marginal_eq = marg_eq(x, B)),
    lapply(split_variants, wy_split, x = x, B = B))
  t(vapply(fl, function(f) {
    c(any = any(f$low | f$high), any_low = any(f$low),
      any_high = any(f$high), low = f$low, high = f$high)
  }, numeric(3 + 2 * k)))
}

grid <- rbind(
  expand.grid(k = c(1, 9, 20), rho = c(0, 0.5), skew = c("0.5", "0.15"),
    stringsAsFactors = FALSE),
  expand.grid(k = c(9, 20), rho = c(0, 0.5), skew = "mixed",
    stringsAsFactors = FALSE)
)

syn_split <- lapply(seq_len(nrow(grid)), function(g) {
  k <- grid$k[g]
  skew <- if (grid$skew[g] == "mixed") {
    seq(0.02, 0.3, length.out = k)
  } else {
    as.numeric(grid$skew[g])
  }
  res <- mclapply(seq_len(REPS), function(r) {
    one_rep(r, k, grid$rho[g], skew)
  }, mc.cores = NCORES)
  list(cond = grid[g, ], counts = Reduce('+', res))
})
saveRDS(syn_split, SPLIT_FILE)
}

syn_split <- readRDS(SPLIT_FILE)

split_tab <- do.call(rbind, lapply(syn_split, function(x) {
  cnt <- x$counts
  data.frame(
    skew = x$cond$skew, k = x$cond$k, rho = x$cond$rho,
    variant = rownames(cnt),
    any = 100 * cnt[, "any"] / REPS,
    any_low = 100 * cnt[, "any_low"] / REPS,
    any_high = 100 * cnt[, "any_high"] / REPS,
    flags_low = rowSums(cnt[, grep("^low", colnames(cnt)), drop = FALSE]),

```

```

    flags_high = rowSums(cnt[, grep("^high", colnames(cnt)), drop = FALSE]),
    row.names = NULL, stringsAsFactors = FALSE
  )
}))

```

Checks

abs_t against the cached synthetic results. All three differences should be 0.

```

cached <- readRDS(SYN_FILE)
names(cached) <- sub("\\.", "_", names(cached))
cached |>
  mutate(skew = as.character(skew)) |>
  select(skew, k, rho, abs_t_any, abs_t_low, abs_t_high) |>
  inner_join(filter(split_tab, variant == "abs_t"),
    by = c("skew", "k", "rho")) |>
  summarise(conditions = n(),
    max_diff_any = max(abs(abs_t_any - any)),
    max_diff_low = max(abs(abs_t_low - flags_low)),
    max_diff_high = max(abs(abs_t_high - flags_high)))

```

conditions	max_diff_any	max_diff_low	max_diff_high
12	0	0	0

With one item, the split variants and the marginal test should give the same lower/upper flag counts.

```

split_tab |>
  filter(k == 1, variant != "abs_t") |>
  transmute(skew, rho, variant, flags = paste0(flags_low, "/", flags_high)) |>
  pivot_wider(names_from = variant, values_from = flags)

```

skew	rho	marginal_eq	split_t	split_two_piece
0.5	0.0	89/117	89/117	89/117
0.5	0.5	100/111	100/111	100/111
0.15	0.0	89/117	89/117	89/117
0.15	0.5	100/111	100/111	100/111

Family-wise rate and tail balance

```

split_tab |>
  filter(variant != "marginal_eq") |>
  transmute(skew, k, rho, variant, any = round(any, 2)) |>
  pivot_wider(names_from = variant, values_from = any)

```

skew	k	rho	abs_t	split_t	split_two_piece
0.5	1	0.0	4.78	5.15	5.15
0.5	9	0.0	5.25	5.25	5.90
0.5	20	0.0	5.25	5.60	5.82
0.5	1	0.5	4.85	5.28	5.28

skew	k	rho	abs_t	split_t	split_two_piece
0.5	9	0.5	4.62	5.00	5.35
0.5	20	0.5	5.15	5.40	5.80
0.15	1	0.0	4.88	5.15	5.15
0.15	9	0.0	5.10	5.38	5.78
0.15	20	0.0	5.15	5.35	5.78
0.15	1	0.5	5.28	5.28	5.28
0.15	9	0.5	4.58	5.08	5.20
0.15	20	0.5	5.18	5.60	5.80
mixed	9	0.0	5.00	5.38	5.95
mixed	20	0.0	5.25	5.28	5.45
mixed	9	0.5	4.88	5.35	5.62
mixed	20	0.5	4.90	5.65	5.68

```
split_tab |>
  filter(variant != "marginal_eq") |>
  transmute(skew, k, rho, variant,
    rate = paste0(round(any_low, 2), " / ", round(any_high, 2))) |>
  pivot_wider(names_from = variant, values_from = rate)
```

skew	k	rho	abs_t	split_t	split_two_piece
0.5	1	0.0	4.78 / 0	2.22 / 2.92	2.22 / 2.92
0.5	9	0.0	5.25 / 0	2.88 / 2.4	2.85 / 3.1
0.5	20	0.0	5.25 / 0	2.9 / 2.78	2.85 / 3.02
0.5	1	0.5	4.85 / 0	2.5 / 2.78	2.5 / 2.78
0.5	9	0.5	4.62 / 0	2.45 / 2.55	2.4 / 2.95
0.5	20	0.5	5.15 / 0	2.72 / 2.67	2.65 / 3.15
0.15	1	0.0	3.38 / 1.5	2.22 / 2.92	2.22 / 2.92
0.15	9	0.0	5.05 / 0.05	2.92 / 2.48	2.85 / 2.98
0.15	20	0.0	5.1 / 0.05	2.78 / 2.65	2.98 / 2.88
0.15	1	0.5	3.52 / 1.75	2.5 / 2.78	2.5 / 2.78
0.15	9	0.5	4.4 / 0.17	2.35 / 2.72	2.38 / 2.83
0.15	20	0.5	5.08 / 0.1	2.72 / 2.88	2.7 / 3.1
mixed	9	0.0	4.82 / 0.17	2.88 / 2.55	3.05 / 2.95
mixed	20	0.0	5.2 / 0.05	2.55 / 2.78	2.78 / 2.78
mixed	9	0.5	4.5 / 0.38	2.5 / 2.85	2.62 / 3
mixed	20	0.5	4.8 / 0.1	2.78 / 2.88	2.7 / 2.98

```
split_tab |>
  filter(variant != "marginal_eq") |>
```

```
transmute(skew, k, rho, variant,
          flags = paste0(flags_low, "/", flags_high)) |>
pivot_wider(names_from = variant, values_from = flags)
```

skew	k	rho	abs_t	split_t	split_two_piece
0.5	1	0.0	191/0	89/117	89/117
0.5	9	0.0	215/0	115/96	116/125
0.5	20	0.0	219/0	119/113	118/125
0.5	1	0.5	194/0	100/111	100/111
0.5	9	0.5	305/0	147/150	144/186
0.5	20	0.5	371/0	171/187	167/230
0.15	1	0.0	135/60	89/117	89/117
0.15	9	0.0	206/2	117/99	116/120
0.15	20	0.0	214/2	114/108	123/118
0.15	1	0.5	141/70	100/111	100/111
0.15	9	0.5	287/15	142/169	139/183
0.15	20	0.5	364/6	166/211	167/225
mixed	9	0.0	197/7	115/104	124/119
mixed	20	0.0	217/2	102/111	113/114
mixed	9	0.5	286/19	142/164	147/186
mixed	20	0.5	337/6	164/196	168/222

The first table is the family-wise rate, nominal 5. The second is the family-wise rate per direction, lower / upper, nominal 2.5 each for the split variants. The third counts flags in the lower and upper tail, summed over replications, in the format of the `synthetic` table.

Balance across items in the mixed arm

Lower/upper flags summed over replications, by thirds of the items from least to most skewed.

```
mixed <- do.call(rbind, lapply(
  Filter(function(x) x$cond$skew == "mixed", syn_split),
  function(x) {
    cnt <- x$counts
    third <- cut(seq_len(x$cond$k), 3, labels = c("least", "middle", "most"))
    lo <- cnt[, grep("^low", colnames(cnt)), drop = FALSE]
    hi <- cnt[, grep("^high", colnames(cnt)), drop = FALSE]
    do.call(rbind, lapply(levels(third), function(t3) data.frame(
      k = x$cond$k, rho = x$cond$rho, variant = rownames(cnt),
      skew_third = t3,
      low = rowSums(lo[, third == t3, drop = FALSE]),
      high = rowSums(hi[, third == t3, drop = FALSE]),
      row.names = NULL, stringsAsFactors = FALSE
    )))
  })
mixed |>
  filter(variant != "marginal_eq") |>
```

```
transmute(k, rho, variant, skew_third,
          flags = paste0(low, "/", high)) |>
pivot_wider(names_from = skew_third, values_from = flags) |>
arrange(k, rho, variant)
```

k	rho	variant	least	middle	most
9	0.0	abs_t	33/6	72/1	92/0
9	0.0	split_t	14/81	36/22	65/1
9	0.0	split_two_piece	35/55	39/39	50/25
9	0.5	abs_t	55/15	90/4	141/0
9	0.5	split_t	16/109	43/40	83/15
9	0.5	split_two_piece	39/78	46/61	62/47
20	0.0	abs_t	28/2	64/0	125/0
20	0.0	split_t	7/89	30/19	65/3
20	0.0	split_two_piece	22/50	35/36	56/28
20	0.5	abs_t	52/6	101/0	184/0
20	0.5	split_t	17/138	53/45	94/13
20	0.5	split_two_piece	38/103	52/64	78/55

Split results. The checks passed: `abs_t` reproduced the cached synthetic results, and with one item the split variants matched the marginal test. Splitting the step-down balanced the two directions. Over the 12 cells with $k \geq 9$, `split_t` flagged at least one item in the lower direction in 2.70 percent of replications and in the upper in 2.68, against 4.94 and 0.09 for $|t|$. In the worst cell of the handoff ($s = .15$, $k = 20$, $p = .5$) the lower/upper flag counts went from 364/6 to 166/211.

Within a direction the imbalance moved to the items. In the mixed arm with 20 correlated items, `split_t` gave the least, middle and most skewed thirds 17, 53 and 94 lower flags, and 138, 45 and 13 upper flags. Two-piece scaling roughly halved that gradient, to 38, 52 and 78 lower and 103, 64 and 55 upper. Both split variants were slightly above the nominal rate, which the next run traces to self-influence.

Pooled moments and a second seed block

Over the 12 cells with $k \geq 9$, `split_t` averaged 5.36 percent and `split_two_piece` 5.68, against 5.03 for $|t|$ on the same replications. Two explanations fit. The conditions share seeds, and on these seeds the one-item marginal test, nominal at 4.99 by construction ($2 \times 10/401$), gave 5.15 and 5.28. Or self-influence: the mean, SD and quantiles come from the simulated values only, so an extreme simulated value enlarges its own scale and shrinks its own statistic, which the observed value never does to itself.

The `_pooled` variants compute the mean, SD and quantiles from all $B + 1$ values. The transform is then the same function of the column whichever row is taken as observed, so the rows stay exchangeable and each one-sided step-down rejects with probability $10/401$ (2.49 percent) by construction. On the same seeds, pooled rates near 2.49 per direction point to self-influence, and pooled rates as high as the simulated-only ones point to seed noise. A second block of 4000 seeds (4001 to 8000) gives a fresh estimate for every variant.

The step-down here is a vectorised copy of `.wy_stepdown()`, which keeps the maximum over the remaining items as a running maximum instead of recomputing it at each step. Block 1 reruns seeds 1 to 4000, so its simulated-only counts must equal the cached `split-run` results, which checks the copy against the package function on every replication.

What to check.

1. Block 1 reproduces the `split-run` counts for `abs_t`, `split_t`, `split_two_piece` and `marginal_eq`.
2. With $k = 1$, all split variants and `marginal_eq` give the same flag counts.
3. Pooled variants at 2.49 percent per direction. The Monte Carlo SE is 0.25 points per block and 0.17 over both blocks.
4. Paired, the replications flagged by one version and not the other. Under self-influence the simulated-only versions should account for nearly all of them.

Results are cached in `restscore_eti_null/synthetic_pooled.rds`.

```
POOL_FILE <- file.path(RES_DIR, "synthetic_pooled.rds")
BLOCKS <- list(block1 = 1:4000, block2 = 4001:8000)
if (!file.exists(POOL_FILE)) {
  ALPHA <- 0.05
  # Same result as .wy_stepdown() when nothing is missing. Items are taken
  # from the smallest observed statistic to the largest, and the maximum over
  # the remaining items is updated with pmax().
  wy_fast <- function(stat_obs, stat_sim, B) {
    k <- length(stat_obs)
    ord <- order(stat_obs, decreasing = TRUE)
    m <- rep(-Inf, nrow(stat_sim))
    p <- numeric(k)
    for (j in k:1) {
      m <- pmax(m, stat_sim[, ord[j]])
      p[j] <- (1 + sum(m >= stat_obs[ord[j]])) / (B + 1)
    }
    padj <- numeric(k)
    padj[ord] <- pmin(cummax(p), 1)
    padj
  }
  # The reference sample is the simulated values, or all B + 1 when pooled.
  t_stat <- function(o, sim, pooled) {
    r <- if (pooled) c(o, sim) else sim
    m <- mean(r); sdv <- stats::sd(r)
    list(o = (o - m) / sdv, s = (sim - m) / sdv)
  }
  tp_stat <- function(o, sim, pooled) {
    r <- if (pooled) c(o, sim) else sim
    q <- stats::quantile(r, c(.025, .5, .975), names = FALSE)
    up <- q[3] - q[2]; dn <- q[2] - q[1]
    list(o_hi = (o - q[2]) / up, s_hi = (sim - q[2]) / up,
         o_lo = (q[2] - o) / dn, s_lo = (q[2] - sim) / dn)
  }
  flags_abs <- function(zo, zs, B) {
    padj <- wy_fast(abs(zo), abs(zs), B)
    list(low = padj < ALPHA & zo < 0, high = padj < ALPHA & zo > 0)
  }
  flags_split <- function(o_hi, s_hi, o_lo, s_lo, B) {
    list(low = wy_fast(o_lo, s_lo, B) < ALPHA / 2,
         high = wy_fast(o_hi, s_hi, B) < ALPHA / 2)
  }
  # Same generator and seeds as `synthetic` and `split-run`.
  one_rep <- function(r, k, rho, skew, B = 400) {
    set.seed(r)
    common <- rnorm(B + 1)
    s <- rep_len(skew, k)
  }
}
```

```

x <- sapply(seq_len(k), function(j) {
  -exp(s[j] * (sqrt(rho) * common + sqrt(1 - rho) * rnorm(B + 1)))
})
x <- matrix(x, B + 1, k)
fl <- list()
for (pooled in c(FALSE, TRUE)) {
  sfx <- if (pooled) "_pooled" else ""
  tt <- lapply(seq_len(k), function(j) t_stat(x[1, j], x[-1, j], pooled))
  zo <- vapply(tt, `[`, numeric(1), "o")
  zs <- matrix(sapply(tt, `[`, "s"), B, k)
  fl[[paste0("abs_t", sfx)]] <- flags_abs(zo, zs, B)
  fl[[paste0("split_t", sfx)]] <- flags_split(zo, zs, -zo, -zs, B)
  tp <- lapply(seq_len(k), function(j) tp_stat(x[1, j], x[-1, j], pooled))
  tp_o <- function(nm) vapply(tp, `[`, numeric(1), nm)
  tp_s <- function(nm) matrix(sapply(tp, `[`, nm), B, k)
  fl[[paste0("split_two_piece", sfx)]] <- flags_split(
    tp_o("o_hi"), tp_s("s_hi"), tp_o("o_lo"), tp_s("s_lo"), B
  )
}
o <- x[1, ]; sm <- x[-1, , drop = FALSE]
fl$marginal_eq <- list(
  low = (1 + colSums(sweep(sm, 2, o, "<="))) / (B + 1) < ALPHA / 2,
  high = (1 + colSums(sweep(sm, 2, o, ">="))) / (B + 1) < ALPHA / 2
)
t(vapply(fl, function(f) {
  c(any = any(f$low | f$high), any_low = any(f$low),
    any_high = any(f$high), low = f$low, high = f$high)
}, numeric(3 + 2 * k)))
}
grid <- rbind(
  expand.grid(k = c(1, 9, 20), rho = c(0, 0.5), skew = c("0.5", "0.15"),
    stringsAsFactors = FALSE),
  expand.grid(k = c(9, 20), rho = c(0, 0.5), skew = "mixed",
    stringsAsFactors = FALSE)
)
syn_pooled <- list()
for (g in seq_len(nrow(grid))) {
  k <- grid$k[g]
  skew <- if (grid$skew[g] == "mixed") {
    seq(0.02, 0.3, length.out = k)
  } else {
    as.numeric(grid$skew[g])
  }
  for (b in names(BLOCKS)) {
    res <- mclapply(BLOCKS[[b]], function(r) {
      one_rep(r, k, grid$rho[g], skew)
    }, mc.cores = NCORES)
    syn_pooled[[length(syn_pooled) + 1]] <- list(
      cond = grid[g, ], block = b, counts = Reduce(`+`, res),
      any = vapply(res, function(m) m[, "any"] == 1,
        logical(nrow(res[[1]])))
    )
  }
}
}

```

```

    saveRDS(syn_pooled, P00L_FILE)
  }
  syn_pooled <- readRDS(P00L_FILE)

  pool_tab <- do.call(rbind, lapply(syn_pooled, function(x) {
    cnt <- x$counts
    reps <- length(BLOCKS[[x$block]])
    data.frame(
      skew = x$cond$skew, k = x$cond$k, rho = x$cond$rho, block = x$block,
      variant = rownames(cnt),
      any = 100 * cnt[, "any"] / reps,
      any_low = 100 * cnt[, "any_low"] / reps,
      any_high = 100 * cnt[, "any_high"] / reps,
      flags_low = rowSums(cnt[, grep("^low", colnames(cnt)), drop = FALSE]),
      flags_high = rowSums(cnt[, grep("^high", colnames(cnt)), drop = FALSE]),
      row.names = NULL, stringsAsFactors = FALSE
    )
  })))

```

Checks

Block 1 against split-run, all count cells of the four shared variants. The difference should be 0.

```

b1 <- Filter(function(x) x$block == "block1", syn_pooled)
data.frame(
  conditions = length(b1),
  same_conditions = all(mapply(function(a, b) {
    identical(a$cond$k, b$cond$k) && identical(a$cond$rho, b$cond$rho) &&
    identical(a$cond$skew, b$cond$skew)
  }, syn_split, b1)),
  max_diff = max(mapply(function(a, b) {
    v <- rownames(a$counts)
    max(abs(a$counts[v, ] - b$counts[v, colnames(a$counts)]))
  }, syn_split, b1))
)

```

conditions	same_conditions	max_diff
16	TRUE	0

With one item, both blocks.

```

pool_tab |>
  filter(k == 1, !grepl("^abs_t", variant)) |>
  transmute(skew, rho, block, variant,
    flags = paste0(flags_low, "/", flags_high)) |>
  pivot_wider(names_from = variant, values_from = flags)

```

skew	who	block	split_t	split_two_piece	split_t_pooled	split_two_piece_pooled	marginal_eq
0.5	0.0	block1	89/117	89/117	89/117	89/117	89/117
0.5	0.0	block2	100/99	100/99	100/99	100/99	100/99
0.5	0.5	block1	100/111	100/111	100/111	100/111	100/111

skew	rho	block	split_t	split_two_piece	split_t_pooled	split_two_piece_pooled	marginal_eq
0.5	0.5	block2	107/95	107/95	107/95	107/95	107/95
0.15	0.0	block1	89/117	89/117	89/117	89/117	89/117
0.15	0.0	block2	100/99	100/99	100/99	100/99	100/99
0.15	0.5	block1	100/111	100/111	100/111	100/111	100/111
0.15	0.5	block2	107/95	107/95	107/95	107/95	107/95

Family-wise rate, simulated-only against pooled

Means over the 12 cells with $k \geq 9$, per block, in percent. Nominal per direction is 2.49 for the split variants.

```
pool_tab |>
  filter(k > 1, variant != "marginal_eq") |>
  group_by(variant, block) |>
  summarise(fwer = round(mean(any), 2), lower = round(mean(any_low), 2),
            upper = round(mean(any_high), 2), .groups = "drop") |>
  pivot_wider(names_from = block, values_from = c(fwer, lower, upper))
```

variant	fwer_block1	fwer_block2	lower_block1	lower_block2	upper_block1	upper_block2
abs_t	5.03	5.18	4.94	5.10	0.09	0.08
abs_t_pooled	4.70	4.81	4.62	4.74	0.08	0.07
split_t	5.36	5.19	2.70	2.71	2.68	2.48
split_t_pooled	4.95	4.82	2.48	2.48	2.50	2.36
split_two_piece	5.68	5.22	2.73	2.69	2.98	2.56
split_two_piece_pooled	5.02	4.61	2.44	2.42	2.60	2.22

Per cell, both blocks together (8000 replications), family-wise rate and lower / upper rate.

```
pool_both <- pool_tab |>
  filter(variant != "marginal_eq") |>
  group_by(skew, k, rho, variant) |>
  summarise(any = mean(any), any_low = mean(any_low),
            any_high = mean(any_high), .groups = "drop") |>
  mutate(skew = factor(skew, levels = c("0.5", "0.15", "mixed"))) |>
  arrange(skew, rho, k)
pool_both |>
  transmute(skew, k, rho, variant, any = round(any, 2)) |>
  pivot_wider(names_from = variant, values_from = any)
```

skew	k	rho	abs_t	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
0.5	1	0.0	4.86	4.86	5.06	5.06	5.06	5.06
0.5	9	0.0	5.14	4.79	4.95	4.50	5.46	4.72
0.5	20	0.0	5.31	4.64	5.61	5.01	5.59	4.74
0.5	1	0.5	4.94	4.94	5.16	5.16	5.16	5.16
0.5	9	0.5	4.85	4.71	5.22	5.03	5.31	4.85
0.5	20	0.5	5.16	4.86	5.39	4.96	5.50	4.88

skew	k	rho	abs_t	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
0.15	1	0.0	4.89	4.84	5.06	5.06	5.06	5.06
0.15	9	0.0	5.01	4.74	5.06	4.68	5.40	4.76
0.15	20	0.0	5.21	4.68	5.38	4.88	5.54	4.81
0.15	1	0.5	5.15	5.09	5.16	5.16	5.16	5.16
0.15	9	0.5	4.81	4.62	5.09	4.90	5.21	4.86
0.15	20	0.5	5.24	4.90	5.39	5.00	5.49	4.89
mixed	9	0.0	4.94	4.68	5.11	4.79	5.59	4.84
mixed	20	0.0	5.46	4.89	5.45	4.93	5.54	4.82
mixed	9	0.5	5.04	4.76	5.34	5.09	5.47	4.90
mixed	20	0.5	5.06	4.84	5.28	4.91	5.31	4.72

```
pool_both |>
  filter(grepl("^split", variant)) |>
  transmute(skew, k, rho, variant,
    rate = paste0(round(any_low, 2), " / ", round(any_high, 2))) |>
  pivot_wider(names_from = variant, values_from = rate)
```

skew	k	rho	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
0.5	1	0.0	2.36 / 2.7	2.36 / 2.7	2.36 / 2.7	2.36 / 2.7
0.5	9	0.0	2.64 / 2.33	2.39 / 2.12	2.71 / 2.81	2.44 / 2.35
0.5	20	0.0	2.92 / 2.74	2.51 / 2.55	2.8 / 2.84	2.39 / 2.39
0.5	1	0.5	2.59 / 2.58	2.59 / 2.58	2.59 / 2.58	2.59 / 2.58
0.5	9	0.5	2.67 / 2.55	2.51 / 2.51	2.6 / 2.71	2.42 / 2.42
0.5	20	0.5	2.78 / 2.61	2.45 / 2.51	2.59 / 2.91	2.36 / 2.51
0.15	1	0.0	2.36 / 2.7	2.36 / 2.7	2.36 / 2.7	2.36 / 2.7
0.15	9	0.0	2.67 / 2.41	2.46 / 2.24	2.71 / 2.75	2.45 / 2.36
0.15	20	0.0	2.8 / 2.64	2.53 / 2.41	2.84 / 2.76	2.44 / 2.41
0.15	1	0.5	2.59 / 2.58	2.59 / 2.58	2.59 / 2.58	2.59 / 2.58
0.15	9	0.5	2.6 / 2.49	2.51 / 2.39	2.59 / 2.62	2.46 / 2.4
0.15	20	0.5	2.7 / 2.69	2.45 / 2.55	2.64 / 2.85	2.36 / 2.53
mixed	9	0.0	2.7 / 2.46	2.45 / 2.38	2.88 / 2.78	2.51 / 2.38
mixed	20	0.0	2.7 / 2.78	2.46 / 2.47	2.91 / 2.69	2.59 / 2.29
mixed	9	0.5	2.65 / 2.69	2.53 / 2.56	2.76 / 2.71	2.45 / 2.45
mixed	20	0.5	2.66 / 2.61	2.46 / 2.45	2.54 / 2.78	2.3 / 2.42

Paired comparison

Replications flagged by the simulated-only version and not the pooled one, and the reverse, summed over the 12 cells with $k \geq 9$ and both blocks.

```
pairs <- c("abs_t", "split_t", "split_two_piece")
do.call(rbind, lapply(syn_pooled, function(x) {
```

```

a <- x$any
data.frame(
  k = x$cond$k, variant = pairs,
  sim_only = vapply(pairs, function(v) {
    sum(a[v, ] & !a[paste0(v, "_pooled"), ])
  }, numeric(1)),
  pooled_only = vapply(pairs, function(v) {
    sum(!a[v, ] & a[paste0(v, "_pooled"), ])
  }, numeric(1)),
  row.names = NULL
)
})) |>
filter(k > 1) |>
group_by(variant) |>
summarise(sim_only = sum(sim_only), pooled_only = sum(pooled_only))

```

variant	sim_only	pooled_only
abs_t	331	0
split_t	368	0
split_two_piece	636	27

Pooled results. Block 1 reproduced the `split-run` counts in every cell, so the vectorised step-down agrees with `.wy_stepdown()`. With one item, block 2 gave 4.98 and 5.05 percent against 5.15 and 5.28 in block 1, so the first seed block ran slightly high.

1. The excess is self-influence. Over 96,000 paired replications (12 cells, two blocks), the simulated-only versions flagged 331 (`abs_t`), 368 (`split_t`) and 636 (`split_two_piece`) replications that their pooled versions did not, against 0, 0 and 27 the other way. Seed noise would give discordance in both directions. The size, about 0.35 points for the t-based statistics, is what one extreme value's pull on its own SD predicts, roughly $(t^2 - 1)/(2B)$ or 1 percent of scale at $t = 3$ and $B = 400$. Quantile-based scales move further, since one value shifts the 97.5th percentile by a whole order statistic.

2. Pooling restores the level and keeps the balance. `split_t_pooled` flagged at least one item in 4.95 and 4.82 percent of replications in the two blocks, with 2.48 and 2.48 percent in the lower direction and 2.50 and 2.36 in the upper, against 2.49 by construction. `split_two_piece_pooled` gave 5.02 and 4.61.

3. Two-piece scaling is the better split. In the mixed arm, summed over its four cells and both blocks, `split_t_pooled` gave the most skewed third of the items 5.9 times the lower flags of the least skewed third, and the least skewed third 19 times the upper flags of the most skewed. For `split_two_piece_pooled` both ratios were 2.1. Pooling changed these ratios little.

4. The shipped `|t|` has the same bias. The package version flagged 331 replications that pooled `|t|` did not, about 0.35 points. Pooled `|t|` came out at 4.70 and 4.81 against 4.99 by construction, within the noise of cells that share seeds, but worth watching on the real null. The validation study found the package `|t|` at 5.0 on the real restscore null, so the bias is small in practice. Pooled moments in `.bootstrap_pvalues()` would remove it for every cutoff function, which is a separate decision.

Decision. `split_two_piece_pooled` is the candidate, with `split_t_pooled` alongside since the two differ in item balance and may differ in power. Both go to the real restscore null.

Real restscore null for the split candidates

The synthetic nulls are exchangeable by construction. The restscore bootstrap is only approximately so, since its null is simulated from parameters estimated on the observed data. This run repeats the real

null of the first half of this document with the split and pooled variants: the four cells and 500 datasets per cell of the validation study, conditional DGP, $B = 400$, and the same seeds. All variants use the same simulated values.

- `abs_t`: the package's `|t|` from `RMitemRestscore()`. It must reproduce `restscore_cutoff_validation/`, and the local copy of its arithmetic (`padj_abs`) must equal it.
- `abs_t_pooled`, `split_t`, `split_t_pooled`, `split_two_piece`, `split_two_piece_pooled`: as in the synthetic sections, with the moment arithmetic of `.bootstrap_pvalues()` (`colMeans()` and `sd()` with `na.rm = TRUE`, zero-variance comparisons set to NA) and `.wy_stepdown()`.

The observed differences and simulated matrices of every dataset are saved in `restscore_split_null/<cell>_sims.rds`, about 35 MB in all, so later variants can be checked without rerunning the bootstrap.

Outcomes. The family-wise rate, nominal 5 percent with a Monte Carlo SE of 0.97 points per cell and 0.49 over all 2000 datasets. The rate per direction, nominal 2.5 for the split variants. The paired comparison of simulated-only and pooled versions. Flags by thirds of the items, ranked by the skewness of their simulated null. With about 25 flags per cell and variant, the last table is descriptive.

Cost. About 2.2 hours on 10 cores, the time of the validation study's conditional arm. The run is off by default. Set `RUN_SPLIT_NULL <- TRUE` to start it. Results are saved per cell, so an interrupted render resumes, and the result chunks run once all four cells are saved.

```
RUN_SPLIT_NULL <- TRUE
SPLIT_NULL_DIR <- "restscore_split_null"
split_null_files <- file.path(SPLIT_NULL_DIR, paste0(cells$cell, ".rds"))
SPLIT_NULL_READY <- all(file.exists(split_null_files))
```

```
# Adjusted p-values of every variant for one dataset, with the arithmetic of
# .bootstrap_pvalues().
split_null_variants <- function(obs, sim) {
  sim <- sim[, names(obs), drop = FALSE]
  B <- nrow(sim)
  guard <- function(x) {
    x[!is.finite(x) | x < 1e-8] <- NA_real_
    x
  }
  out <- data.frame(item = names(obs), stringsAsFactors = FALSE)
  for (pooled in c(FALSE, TRUE)) {
    sfx <- if (pooled) "_pooled" else ""
    ref <- if (pooled) rbind(obs, sim) else sim
    m <- colMeans(ref, na.rm = TRUE)
    s <- guard(apply(ref, 2L, stats::sd, na.rm = TRUE))
    t_obs <- (obs - m) / s
    t_sim <- sweep(sweep(sim, 2L, m, `-`), 2L, s, `/`)
    if (!pooled) out$null_skew <- colMeans(t_sim^3, na.rm = TRUE)
    q <- apply(ref, 2L, stats::quantile, probs = c(.025, .5, .975),
              na.rm = TRUE, names = FALSE)
    up <- guard(q[3, ] - q[2, ])
    dn <- guard(q[2, ] - q[1, ])
    dev_sim <- sweep(sim, 2L, q[2, ], `-`)
    out[[paste0("t", sfx)]] <- t_obs
    out[[paste0("padj_abs", sfx)]] <- .wy_stepdown(abs(t_obs), abs(t_sim), B)
    out[[paste0("padj_t_hi", sfx)]] <- .wy_stepdown(t_obs, t_sim, B)
    out[[paste0("padj_t_lo", sfx)]] <- .wy_stepdown(-t_obs, -t_sim, B)
    out[[paste0("padj_tp_hi", sfx)]] <- .wy_stepdown(
```

```

      (obs - q[2, ]) / up, sweep(dev_sim, 2L, up, `/'`), B
    )
    out[[paste0("padj_tp_lo", sfx)]] <- .wy_stepdown(
      (q[2, ] - obs) / dn, sweep(-dev_sim, 2L, dn, `/'`), B
    )
  }
  out
}

# Same data, seed and cutoff call as one_dataset() and the validation study.
one_dataset_split <- function(rep, cell_row) {
  thr <- item_sets[[cell_row$set]]
  seed <- 1e6 * match(cell_row$cell, cells$cell) + rep
  set.seed(seed)
  repeat {
    d <- as.data.frame(sim_pcm(rnorm(cell_row$n, cell_row$mu, THETA_SD), thr))
    names(d) <- paste0("I", seq_along(thr))
    full <- all(mapply(function(x, m) all(tabulate(x + 1, m + 1) > 0),
                      d, lengths(thr)))
    if (full) break
  }

  tryCatch({
    co <- suppressMessages(RMitemRestscoreCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile", dgp = "conditional"
    ))
    fw <- suppressMessages(RMitemRestscore(
      d, cutoff = co, correction = "fwer", output = "dataframe"
    ))
    sim_mat <- tapply(co$results$Difference,
                      list(co$results$iteration, co$results$Item),
                      function(x) x[1L])
    obs <- stats::setNames(fw$Difference, fw$Item)
    list(
      table = data.frame(
        rep = rep, difference = obs, p_pkg = fw$p_restscore,
        padj_pkg = fw$padj_restscore, split_null_variants(obs, sim_mat),
        actual_iterations = co$actual_iterations,
        row.names = NULL, stringsAsFactors = FALSE
      ),
      sims = list(rep = rep, obs = obs,
                  sim = sim_mat[, names(obs), drop = FALSE])
    ),
    error = function(e) {
      list(error = data.frame(rep = rep, error = conditionMessage(e)))
    }
  })
}

```

```

dir.create(SPLIT_NULL_DIR, showWarnings = FALSE)
for (i in seq_len(nrow(cells))) {
  if (file.exists(split_null_files[i])) next
  t0 <- Sys.time()

```

```

res <- mclapply(seq_len(R_DATASETS),
               function(r) one_dataset_split(r, cells[i, ]),
               mc.cores = NCORES, mc.preschedule = FALSE)
# A worker that dies returns a try-error, kept as an error row.
res <- lapply(res, function(x) {
  if (is.list(x)) x else list(error = data.frame(rep = NA, error = as.character(x)))
})
ok <- vapply(res, function(x) is.null(x$error), logical(1))
# Simulated values first, so a saved result implies saved simulations.
saveRDS(lapply(res[ok], `[[`, "sims"),
        file.path(SPLIT_NULL_DIR, paste0(cells$cell[i], "_sims.rds")))
saveRDS(list(res = lapply(res, function(x) {
  if (is.null(x$error)) x$table else x$error
}),
        elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
        split_null_files[i])
}
SPLIT_NULL_READY <- all(file.exists(split_null_files))

```

```

split_raw <- lapply(cells$cell, function(cl) {
  x <- readRDS(file.path(SPLIT_NULL_DIR, paste0(cl, ".rds")))
  is_ok <- vapply(x$res, function(r) "item" %in% names(r), logical(1))
  list(ok = cbind(cell = cl, do.call(rbind, x$res[is_ok])),
       err = if (any(!is_ok)) cbind(cell = cl, do.call(rbind, x$res[!is_ok])),
       minutes = round(x$elapsed_min, 1))
})
split_null <- do.call(rbind, lapply(split_raw, `[[`, "ok"))
split_null$cell <- factor(split_null$cell, levels = cells$cell)
split_errors <- do.call(rbind, lapply(split_raw, `[[`, "err"))

data.frame(
  cell = cells$cell,
  datasets = vapply(split_raw, function(x) length(unique(x$ok$rep)),
                    integer(1)),
  errors = vapply(split_raw, function(x) {
    if (is.null(x$err)) 0L else nrow(x$err)
  }, integer(1)),
  minutes = vapply(split_raw, `[[`, numeric(1), "minutes")
)

```

cell	datasets	errors	minutes
dich_mis_150	500	0	25.2
dich_tgt_300	500	0	31.8
poly_tgt_150	500	0	25.4
poly_tgt_1000	500	0	45.8

```

if (!is.null(split_errors)) count(split_errors, cell, error)

```

Checks

The package `|t|` against the validation study's conditional arm, and the local copy of its arithmetic against the package. All differences should be 0.

```

val_split <- do.call(rbind, lapply(cells$cell, function(cl) {
  x <- readRDS(file.path(VAL_DIR, paste0(cl, "_conditional.rds")))
  cbind(cell = cl, do.call(rbind, Filter(Negate(is.null), x$res)))
}))
inner_join(
  split_null |> mutate(cell = as.character(cell)) |>
    select(cell, rep, item, difference, p_pkg, padj_pkg, padj_abs),
  val_split |> select(cell, rep, item, difference_val = difference,
    p_val = p, padj_val = padj_fwer),
  by = c("cell", "rep", "item")
) |>
group_by(cell) |>
summarise(items = n(),
  max_diff_difference = max(abs(difference - difference_val)),
  max_diff_p = max(abs(p_pkg - p_val), na.rm = TRUE),
  max_diff_padj = max(abs(padj_pkg - padj_val), na.rm = TRUE),
  max_diff_local = max(abs(padj_abs - padj_pkg), na.rm = TRUE),
  .groups = "drop")

```

cell	items	max_diff_difference	max_diff_p	max_diff_padj	max_diff_local
dich_mis_150	10000	0	0	0	0
dich_tgt_300	10000	0	0	0	0
poly_tgt_1000	4500	0	0	0	0
poly_tgt_150	4500	0	0	0	0

Family-wise rate and tail balance

```

yes <- function(x) !is.na(x) & x
split_variant_levels <- c("abs_t", "abs_t_pooled", "split_t",
  "split_t_pooled", "split_two_piece",
  "split_two_piece_pooled")
split_flags <- split_null |>
  transmute(
    cell, rep, item, null_skew,
    abs_t_low = yes(padj_abs < .05 & t < 0),
    abs_t_high = yes(padj_abs < .05 & t > 0),
    abs_t_pooled_low = yes(padj_abs_pooled < .05 & t_pooled < 0),
    abs_t_pooled_high = yes(padj_abs_pooled < .05 & t_pooled > 0),
    split_t_low = yes(padj_t_lo < .025),
    split_t_high = yes(padj_t_hi < .025),
    split_t_pooled_low = yes(padj_t_lo_pooled < .025),
    split_t_pooled_high = yes(padj_t_hi_pooled < .025),
    split_two_piece_low = yes(padj_tp_lo < .025),
    split_two_piece_high = yes(padj_tp_hi < .025),
    split_two_piece_pooled_low = yes(padj_tp_lo_pooled < .025),
    split_two_piece_pooled_high = yes(padj_tp_hi_pooled < .025)
  ) |>
  pivot_longer(-c(cell, rep, item, null_skew),
    names_to = c("variant", "direction"),
    names_pattern = "(.*)_(low|high)", values_to = "flag") |>
  mutate(variant = factor(variant, levels = split_variant_levels))

```

```

split_fw <- split_flags |>
  group_by(cell, rep, variant) |>
  summarise(any = any(flag), low = any(flag & direction == "low"),
            high = any(flag & direction == "high"), .groups = "drop")
split_fw_summary <- bind_rows(
  split_fw |> mutate(cell = as.character(cell)),
  split_fw |> mutate(cell = "all")
) |>
  mutate(cell = factor(cell, levels = c(cells$cell, "all"))) |>
  group_by(cell, variant) |>
  summarise(datasets = n(), fwer = round(100 * mean(any), 2),
            rate = paste0(round(100 * mean(low), 2), " / ",
                          round(100 * mean(high), 2)),
            .groups = "drop")
split_fw_summary |>
  select(cell, datasets, variant, fwer) |>
  pivot_wider(names_from = variant, values_from = fwer)

```

cell	datasets	abs_t	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
dich_mis_150	500	4.2	4.0	5.2	4.8	7.0	5.4
dich_tgt_300	500	6.8	6.4	8.0	7.6	8.0	6.8
poly_tgt_150	500	3.4	3.0	3.0	2.8	3.6	2.6
poly_tgt_1000	500	5.6	5.0	5.8	4.8	5.4	4.8
all	2000	5.0	4.6	5.5	5.0	6.0	4.9

```

split_fw_summary |>
  select(cell, variant, rate) |>
  pivot_wider(names_from = variant, values_from = rate)

```

cell	abs_t	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
dich_mis_150	3.8 / 0.4	3.6 / 0.4	1.8 / 3.4	1.6 / 3.2	2.6 / 4.4	2 / 3.4
dich_tgt_300	5 / 1.8	4.6 / 1.8	3.6 / 4.6	3.2 / 4.6	3.8 / 4.2	3 / 3.8
poly_tgt_150	2.6 / 1	2.6 / 0.6	1.8 / 1.2	1.6 / 1.2	2 / 1.6	1.4 / 1.2
poly_tgt_1000	3.4 / 3	3 / 2.8	3.4 / 2.8	3.2 / 2	3.6 / 2.2	3.2 / 1.8
all	3.7 / 1.55	3.45 / 1.4	2.65 / 3	2.4 / 2.75	3 / 3.1	2.4 / 2.55

The first table is the family-wise rate, nominal 5. The second is the rate per direction, lower / upper, nominal 2.5 each for the split variants.

Paired comparison

Datasets flagged by the simulated-only version and not the pooled one, and the reverse, over all four cells.

```

split_pw <- split_fw |>
  select(cell, rep, variant, any) |>
  pivot_wider(names_from = variant, values_from = any)
split_pairs <- c("abs_t", "split_t", "split_two_piece")
data.frame(
  variant = split_pairs,
  sim_only = vapply(split_pairs, function(v) {
    sum(split_pw[[v]] & !split_pw[[paste0(v, "_pooled")]])
  }, integer(1)),
  pooled_only = vapply(split_pairs, function(v) {
    sum(!split_pw[[v]] & split_pw[[paste0(v, "_pooled")]])
  }, integer(1)),
  row.names = NULL
)

```

variant	sim_only	pooled_only
abs_t	8	0
split_t	10	0
split_two_piece	22	0

Balance across items

Items are ranked within each cell by the mean skewness of their simulated null, most negative first, and split into thirds. The first table gives the range of skewness, the second the lower/upper flags summed over datasets.

```

split_thirds <- split_null |>
  group_by(cell, item) |>
  summarise(skew = mean(null_skew, na.rm = TRUE), .groups = "drop") |>
  group_by(cell) |>
  mutate(skew_third = cut(rank(skew), 3,
                           labels = c("most", "middle", "least"))) |>
  ungroup()
split_thirds |>
  group_by(cell) |>
  summarise(items = n(), min_skew = round(min(skew), 2),
            mean_skew = round(mean(skew), 2), max_skew = round(max(skew), 2))

```

cell	items	min_skew	mean_skew	max_skew
dich_mis_150	20	-0.69	-0.29	-0.15
dich_tgt_300	20	-0.18	-0.14	-0.12
poly_tgt_150	9	-0.12	-0.09	-0.07
poly_tgt_1000	9	-0.05	-0.04	-0.02

```

split_flags |>
  inner_join(select(split_thirds, cell, item, skew_third),
            by = c("cell", "item")) |>
  group_by(cell, variant, skew_third, direction) |>
  summarise(n = sum(flag), .groups = "drop") |>
  pivot_wider(names_from = direction, values_from = n) |>

```

```
transmute(cell, variant, skew_third, flags = paste0(low, "/", high)) |>
pivot_wider(names_from = skew_third, values_from = flags) |>
select(cell, variant, most, middle, least) |>
arrange(cell, variant)
```

cell	variant	most	middle	least
dich_mis_150	abs_t	8/0	9/0	2/2
dich_mis_150	abs_t_pooled	8/0	8/0	2/2
dich_mis_150	split_t	4/1	3/7	2/9
dich_mis_150	split_t_pooled	3/1	3/7	2/8
dich_mis_150	split_two_piece	3/2	7/8	3/13
dich_mis_150	split_two_piece_pooled	3/2	6/7	1/9
dich_tgt_300	abs_t	8/3	6/5	11/2
dich_tgt_300	abs_t_pooled	8/3	5/5	10/2
dich_tgt_300	split_t	8/7	3/6	7/11
dich_tgt_300	split_t_pooled	7/7	3/6	6/11
dich_tgt_300	split_two_piece	8/7	3/5	8/9
dich_tgt_300	split_two_piece_pooled	6/6	3/5	6/8
poly_tgt_150	abs_t	1/2	5/1	7/2
poly_tgt_150	abs_t_pooled	1/2	5/0	7/1
poly_tgt_150	split_t	0/3	3/2	6/1
poly_tgt_150	split_t_pooled	0/3	3/2	5/1
poly_tgt_150	split_two_piece	1/3	3/4	6/1
poly_tgt_150	split_two_piece_pooled	0/2	3/3	4/1
poly_tgt_1000	abs_t	5/6	8/3	4/6
poly_tgt_1000	abs_t_pooled	5/5	7/3	3/6
poly_tgt_1000	split_t	5/4	8/4	4/6
poly_tgt_1000	split_t_pooled	5/3	7/3	4/4
poly_tgt_1000	split_two_piece	5/3	9/2	4/6
poly_tgt_1000	split_two_piece_pooled	5/2	8/1	3/6